ÉTS
Le génie pour l'industrie

UQÀM | Département d'informatique

Département de génie logiciel et des TI

Chapitre 4 – La culture qualité, les neurosciences, l'éthique et les considérations environnementales des projets logiciels

Professeur Alain April, ing. PHD,
Avec la collaboration de Sylvie Trudel, PhD, (UQAM)
Mis à jour par Bruel Gérançon, PhD, UQAM

> Nous n'avons jamais le temps de faire un travail correctement la première fois, mais nous arrivons toujours à trouver du temps pour refaire le travail une ou deux fois.
>
> Anonyme



Le contenu de ce document est protégé par le droit d'auteur. Ce document peut être utilisé à des fins d'étude personnelle seulement. Aucune partie de cette œuvre ne peut être transmise ou reproduite d'une quelconque façon sans la permission du détenteur du droit d'auteur.



Chapitre 4

4.1 Introduction

4.2 Le coût de la qualité

4.3 La culture qualité

4.4 Les cinq dimensions d'un projet logiciel

© Alain April, 2020 - Tous droits réservés

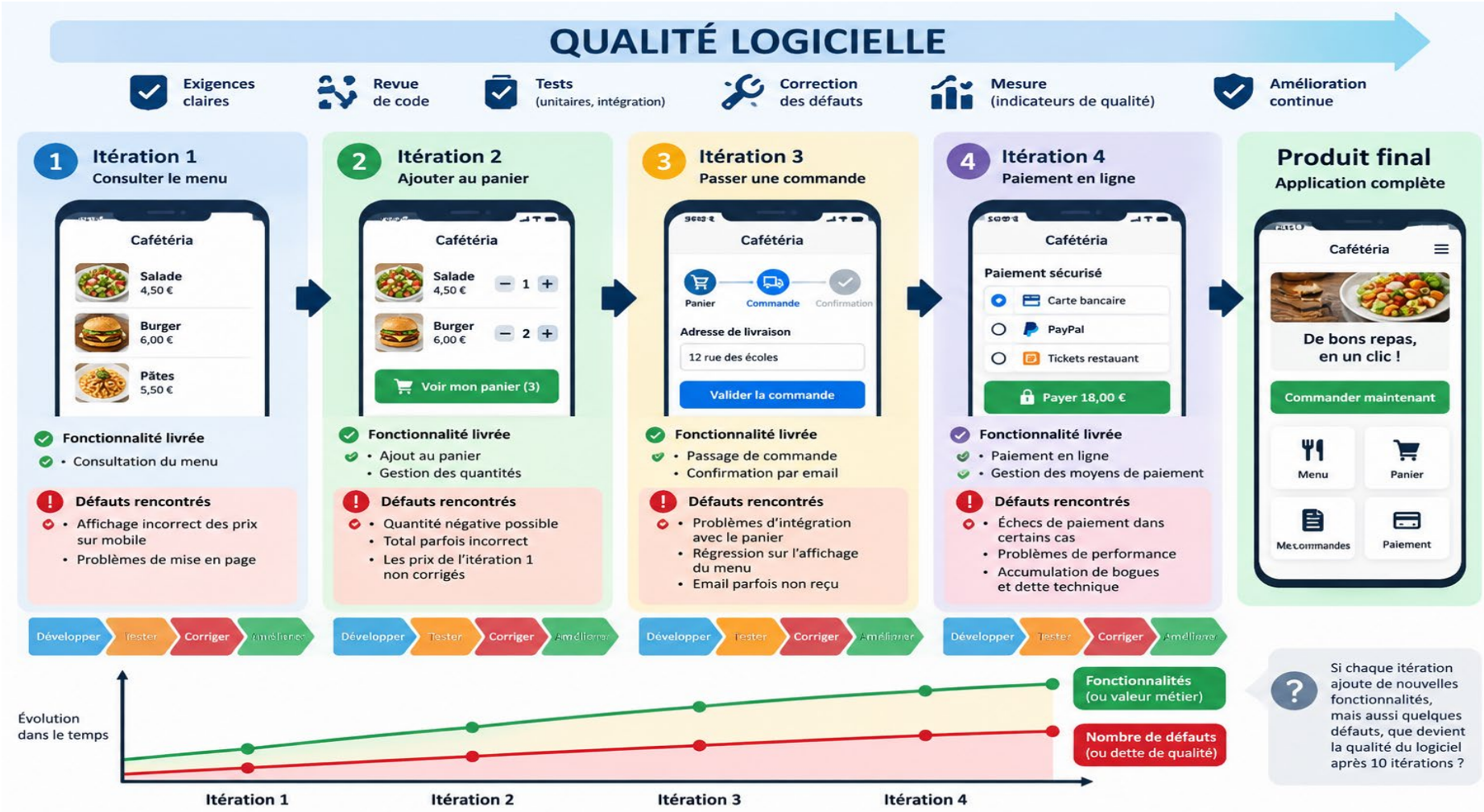
4.1 Introduction

- Ce chapitre est une mise en contexte des enjeux de qualité dans le domaine du logiciel.
  - Coût de la qualité;
  - Culture qualité;

- Pour l'ingénieur membre de son ordre professionnel, l'importance de la qualité fait partie de la responsabilité civile de sa profession et doit être gérée avec soin.

© Alain April, 2020 - Tous droits réservés

Introduction



Introduction

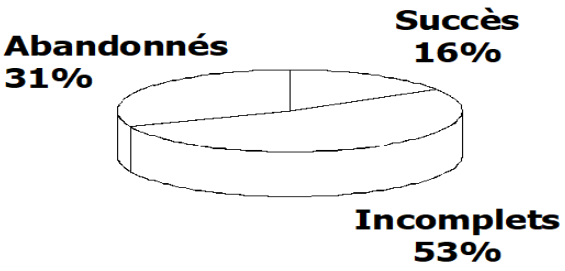
- La qualité logicielle ne doit pas être vérifiée uniquement à la fin du projet.
- Elle doit être construite, mesurée et contrôlée tout au long des itérations.
- Livrer davantage de fonctionnalités ne signifie pas nécessairement produire un meilleur logiciel. L'objectif est de faire évoluer le produit tout en maîtrisant sa qualité à chaque itération.

Échec des projets logiciels

- Échec des projets logiciels

Rapport Chaos *

% des 250 milliards dépensés en développement de logiciels aux États-Unis



Succès = Respect du budget du calendrier et avec toutes les fonctions prévues

Incomplet = Dépassement du budget, du calendrier et fonctions incomplètes

Abandonné = abandonné en cours de développement

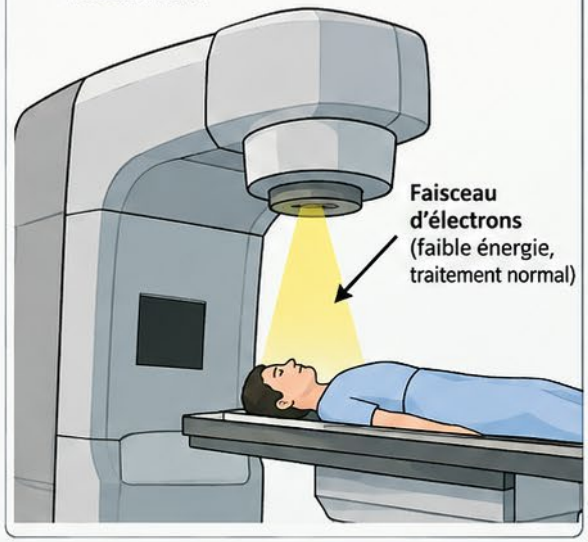
* Plus de 8000 applications étudiées par the Standish Group International, Dennis, MA (USA), 1995.

Le cas du Therac-25

Un défaut logiciel a conduit à l'administration d'une dose massive de rayons, causant la mort de plusieurs patients.

1 Traitement normal

Le Therac-25 est une machine de radiothérapie qui utilise des faisceaux d'électrons à faible énergie pour traiter des cancers.



2 Le défaut logiciel

Lors d'une utilisation rapide (changement de mode et de paramètres), un **bug** du logiciel pouvait laisser la machine dans un état incohérent.

L'opérateur change le mode de traitement très rapidement (ex. : électrons → rayons X)

↓

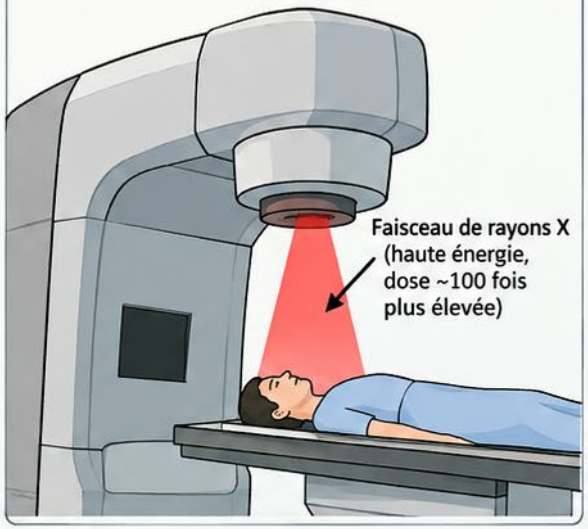
Le logiciel ne met pas correctement à jour certaines variables internes (à cause d'une condition de course entre tâches concurrentes).

↓

La machine croit que les sécurités sont en place… alors qu'elles ne le sont pas réellement.

3 Ce qui se passe

La machine produit alors, au lieu du petit faisceau d'électrons, un puissant faisceau de rayons X (haute énergie), délivrant une dose massive en quelques secondes.



4 Schéma simplifié du défaut

Commande de l'opérateur (changement de mode très rapide) → **Logiciel** (multiples tâches en parallèle)

- **Bug** : état incohérent (certaines variables non mises à jour)
- Vérifications de sécurité contournées ou invalidées

→ **Production du faisceau de rayons X à haute énergie** → **Dose massive au patient** (mort ou blessures graves)

5 Conséquences



- Au moins 5 à 6 patients sont morts entre 1985 et 1987.
- D'autres ont subi des blessures graves (brûlures, nécroses).
- Le problème était dû à un défaut **de conception logicielle** (absence de synchronisation correcte entre tâches).
- Ce cas a marqué l'histoire de la sûreté des logiciels et a conduit à des règles plus strictes pour les logiciels médicaux (validation, gestion des risques, etc.).

Cas de Thérac 25

Causes d'erreurs

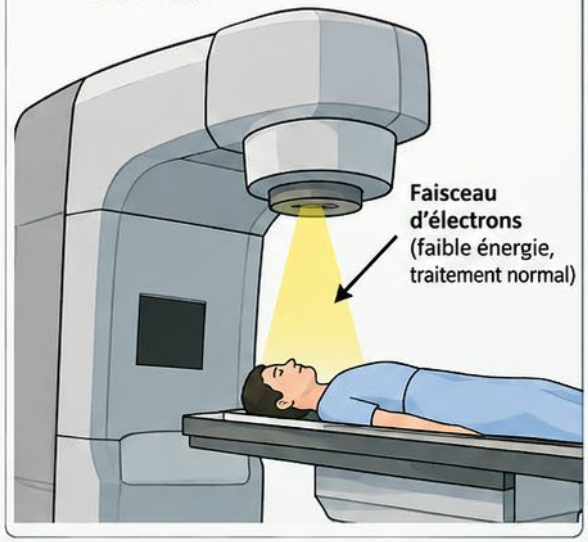
- Code source non évalué par une équipe ou un organisme indépendant
- Étapes liées au test logiciel négligées
- Système mal documenté (la documentation du système n'explique pas les codes indiquant des erreurs)
- Le personnel de l'AECL, au début, n'avait pas tenu compte des plaintes erreurs détectées
- D'autres problèmes liés à la conception

Le cas du Therac-25

Un défaut logiciel a conduit à l'administration d'une dose massive de rayons, causant la mort de plusieurs patients.

1 Traitement normal

Le Therac-25 est une machine de radiothérapie qui utilise des faisceaux d'électrons à faible énergie pour traiter des cancers.



2 Le défaut logiciel

Lors d'une utilisation rapide (changement de mode et de paramètres), un **bug** du logiciel pouvait laisser la machine dans un état incohérent.

L'opérateur change le mode de traitement très rapidement (ex. : électrons → rayons X)

↓

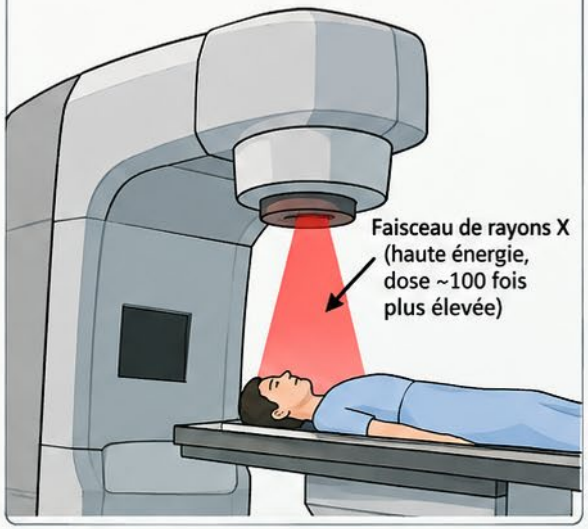
Le logiciel ne met pas correctement à jour certaines variables internes (à cause d'une condition de course entre tâches concurrentes).

↓

La machine croit que les sécurités sont en place… alors qu'elles ne le sont pas réellement.

3 Ce qui se passe

La machine produit alors, au lieu du petit faisceau d'électrons, un puissant faisceau de rayons X (haute énergie), délivrant une dose massive en quelques secondes.



4 Schéma simplifié du défaut

Commande de l'opérateur (changement de mode très rapide) → **Logiciel** (multiples tâches en parallèle)

- **Bug** : état incohérent (certaines variables non mises à jour)
- Vérifications de sécurité contournées ou invalidées

→ **Production du faisceau de rayons X à haute énergie** → **Dose massive au patient** (mort ou blessures graves)

5 Conséquences



- Au moins 5 à 6 patients sont morts entre 1985 et 1987.
- D'autres ont subi des blessures graves (brûlures, nécroses).
- Le problème était dû à un défaut **de conception logicielle** (absence de synchronisation correcte entre tâches).
- Ce cas a marqué l'histoire de la sûreté des logiciels et a conduit à des règles plus strictes pour les logiciels médicaux (validation, gestion des risques, etc.).

Le cas du Boeing 737 MAX

Un défaut logiciel dans le système de contrôle de vol (MCAS) a contribué à la perte de contrôle de l'avion, causant la mort de centaines de personnes.

1 Fonctionnement normal

Le Boeing 737 MAX utilise de nombreux systèmes, dont le système MCAS (Maneuvering Characteristics Augmentation System), qui peut s'activer automatiquement pour éviter un risque de décrochage en abaissant légèrement le nez de l'avion.



2 Le défaut logiciel

À la suite d'une mauvaise conception, le MCAS se base sur une seule sonde d'angle d'attaque (AoA). Si cette sonde fournit une donnée erronée, le logiciel croit que l'avion est en décrochage, même si ce n'est pas le cas.

Sonde AoA défectueuse
(fournit un angle d'attaque trop élevé)

↓

Le logiciel MCAS interprète à tort que l'avion est en décrochage

↓

Le MCAS commande automatiquement l'abaisment répété du nez de l'avion (piqué)

↓

Les pilotes n'ont pas toujours l'information claire pour comprendre l'origine du problème et le contrer efficacement.

3 Ce qui se passe

Le MCAS s'active de façon répétitive, abaissant le nez de l'avion. Malgré les actions des pilotes pour relever le nez, le système continue de se réactiver, ce qui rend l'avion difficile, voire impossible à contrôler.



4 Schéma simplifié du défaut

Sonde AoA
(angle d'attaque)
(donnée erronée)



→ **Logiciel MCAS**
(utilise une seule sonde, sans validation croisée)



→ **Interprétation incorrecte**
(croit qu'il y a un risque de décrochage)



→ **Commande automatique à piquer répétée**



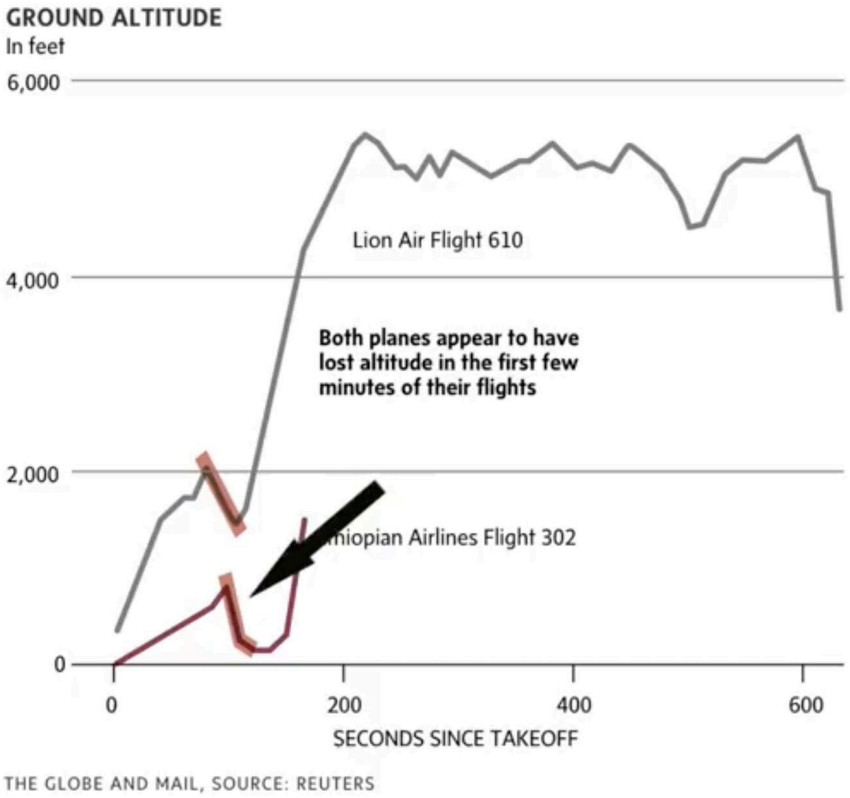
- Abaissement du nez de l'avion
- Réactivation plusieurs fois

5 Conséquences



- **Deux accidents majeurs :**
  - **Lion Air** (29 octobre 2018) : 189 morts
  - Ethiopian Airlines (10 mars 2019) : 157 morts
  (Total : **346** décès)
- **Perte de confiance** dans Boeing et dans la certification.
- Problème dû à un défaut de conception logicielle et à une évaluation insuffisante des risques.
- Nécessité de correctifs logiciels, de formations supplémentaires et de changements dans les processus de certification.

Enquête Boeing 737 Max





Boeing Chief to Families Of Crash Victims : « We Are Sorry, Deeply And truly », NPR, 29 oct 2019

© Alain April, 2020 - Tous droits réservés

Qualité de logiciels et perte de vies humaines

Facteurs contributifs

- Des défauts de conception
- Lacunes dans la formation des pilotes
  - Pas de formation sur simulateur de vol, mais sur une tablette;
  - Pas d'indication claire de la solution dans le manuel de vol
  - Le pilote de Lion Air tentait de trouver une solution dans la documentation lors du crash.
- Des messages d'avertissement ignorés:
  - Déposition d'une plainte officielle et appelle le directeur général concernant la sécurité de l'avion;
  - « Mettrais-tu ta famille dans un avion testé sur simulateur Boeing 737 MAX? Moi, je ne le ferais pas », écrit l'un d'eux en février, huit mois avant la première tragédie.

Leçons apprises - 1

- Conception défensive très faible
  - Ce logiciel ne comprend pas d'auto vérification, de détections d'erreurs et de dispositifs permettant de gérer l'erreur clairement
- **Confusion entre fiabilité et sécurité**
  - « Ce modèle d'avion est très fiable, puisqu'il a fonctionné des milliers de fois sans créer de problème! »;
  - BOEING *assume* que leur logiciel est sécuritaire parce que fiable.

© Alain April, 2020 - Tous droits réservés

Fiabilité & sécurité

- Différence entre **Fiabilité & sécurité?**

Fiabilité & sécurité

Fiabilité

- Capacité d'un logiciel à fonctionner correctement et sans défaillance dans des conditions spécifiques sur une période donnée.
- Minimiser les erreurs et les pannes assurant que le système fonctionne comme prévu

Mesures

- Temps moyen entre pannes
- taux de défaillance,
- Tests de performance.

Fiabilité & sécurité

Sécurité

- Protection contre les
  - accès non autorisés,
  - cyberattaques
  - fuites de données.

But

- Protéger la confidentialité, l'intégrité et la disponibilité des données et des systèmes.

Mesures :

- Analyse de vulnérabilités, audits de sécurité, etc.

© Alain April, 2020 - Tous droits réservés

Leçons apprises - 2

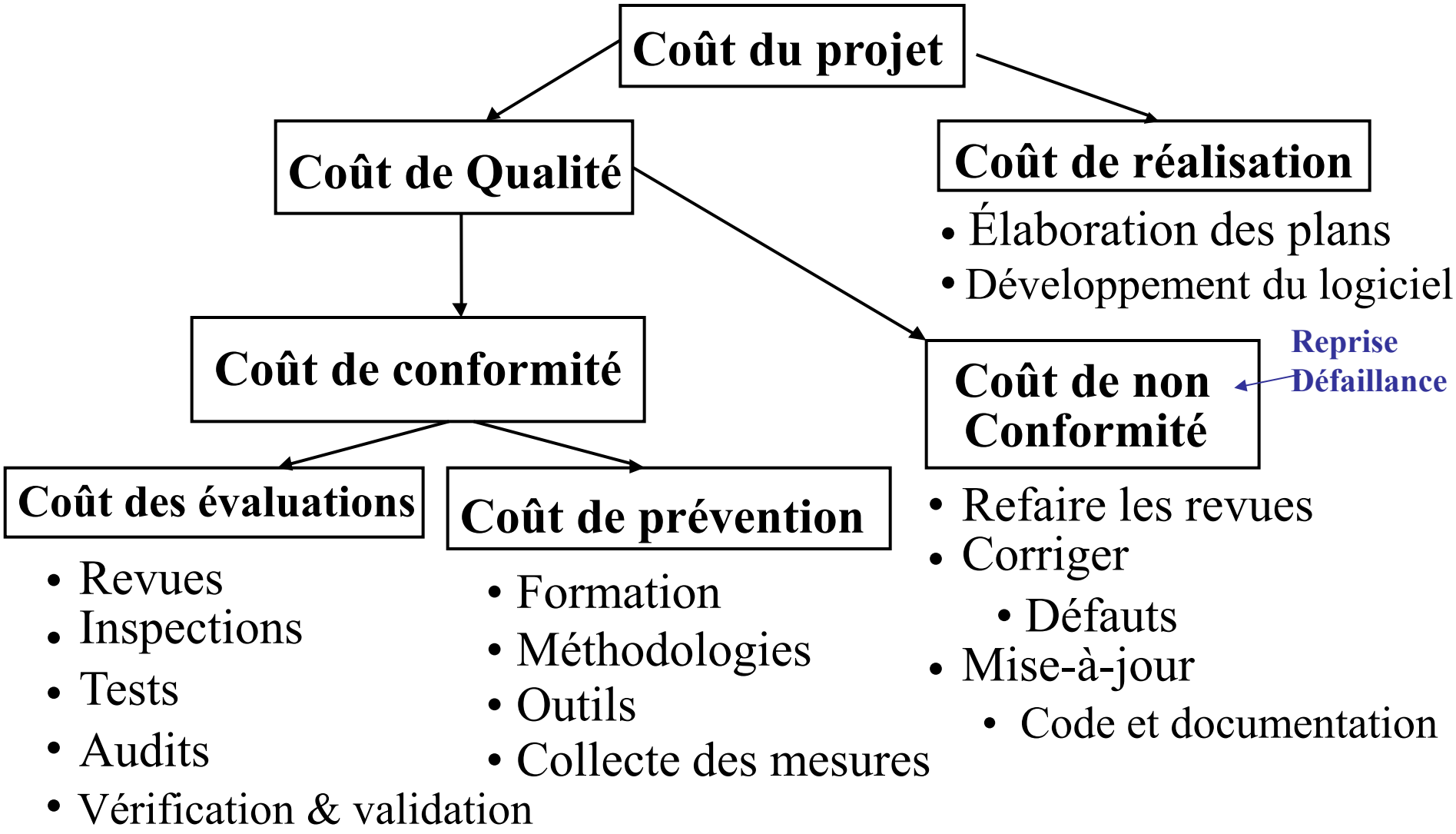
- **Gestionnaires arrogants et incompétents:**
  - Forcer un produit rapidement (perte de marché à cause de la concurrence);
  - Négligence des principes de gestion en technologie;
  - Mépris de l'expertise des séniors (sous-traitance en…. à $9);
  - N'avoir jamais écouté les craintes émises par les employés.

4.2 – Le coût de la qualité

- **Les composants du coût de la qualité**
  - **Coût de prévention**
    - Pour prévenir l'occurrence d'erreurs dans les diverses étapes durant le processus de développement ou de maintenance.
  - **Coût de détection**
    - Le coût de vérification ou d'évaluation d'un produit ou d'un service pendant les différentes étapes du processus de développement.
  - **Coût de défaillance (non-conformité, reprise)**
    - **Coût de défaillance interne**
      - Le coût résultant des anomalies avant que le produit ou le service ne soit livré au client.
    - **Coût de défaillance externe**
      - Le coût encouru par la compagnie quand c'est le client qui découvre des défauts.
    - **Coût du préjudice commercial**
      - Le coût de dégradation de l'image de l'entreprise, coût de la perte de clients.

© Alain April, 2020 - Tous droits réservés

Coût d'un projet



Coût du projet = réalisation + défaillance + évaluation + prévention

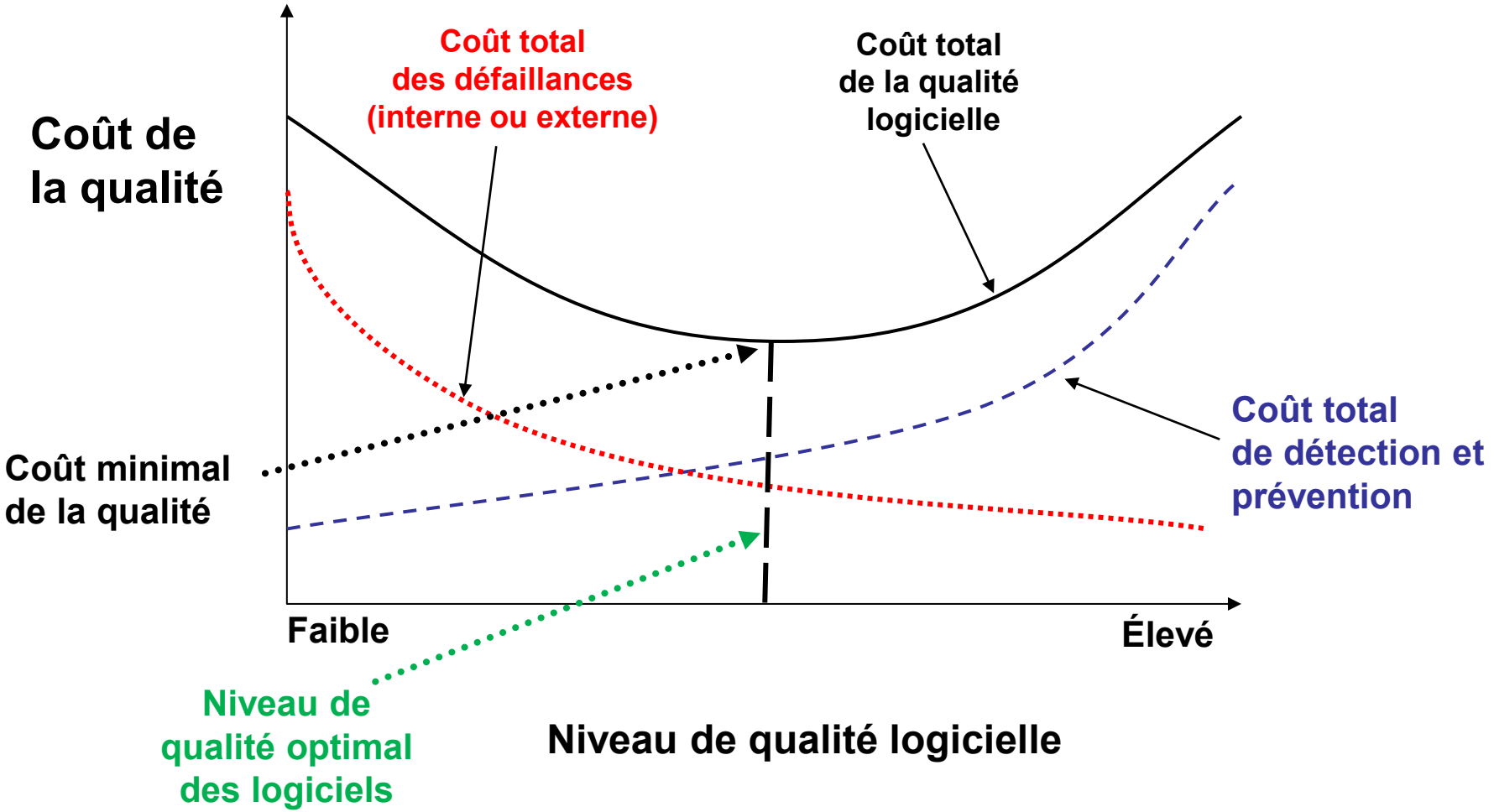
© Alain April, 2020 - Tous droits réservés

Catégories des coûts de la qualité du logiciel

Catégorie majeure	Sous-catégorie	Définition	Coût élémentaire typique
Coûts de prévention	Établir les fondements de la qualité	Efforts pour définir la qualité, établissement d'objectifs de qualité, de normes et de seuil de qualité. Analyse de compromis liés à la qualité	Définition de critères de succès des tests d'acceptation et de normes reliées à la qualité
	Interventions orientées projet et processus	Efforts pour prévenir la mauvaise qualité ou améliorer le processus de qualité	Formation, améliorations de processus, collecte de mesures et analyse
Coûts de détection	Découvrir l'état du produit	Découvrir le niveau de la non-conformité	Test, assurance qualité logicielle, inspections, revues
	S'assurer de l'atteinte de la qualité	Mécanismes de contrôle de la qualité	Audits de qualité du produit, décision de livraison de nouvelles versions
Coûts de défaillances	Anomalie interne	Problèmes de qualité détectés avant la livraison au client	La gestion des défauts avant la livraison, les reprises, les revues et les tests à effectuer à nouveau
	Anomalie externe	Problèmes de qualité détectés après la livraison au client	Support technique des versions livrées, enquête de plaintes, avis de défaut, mise à niveau et correction des erreurs

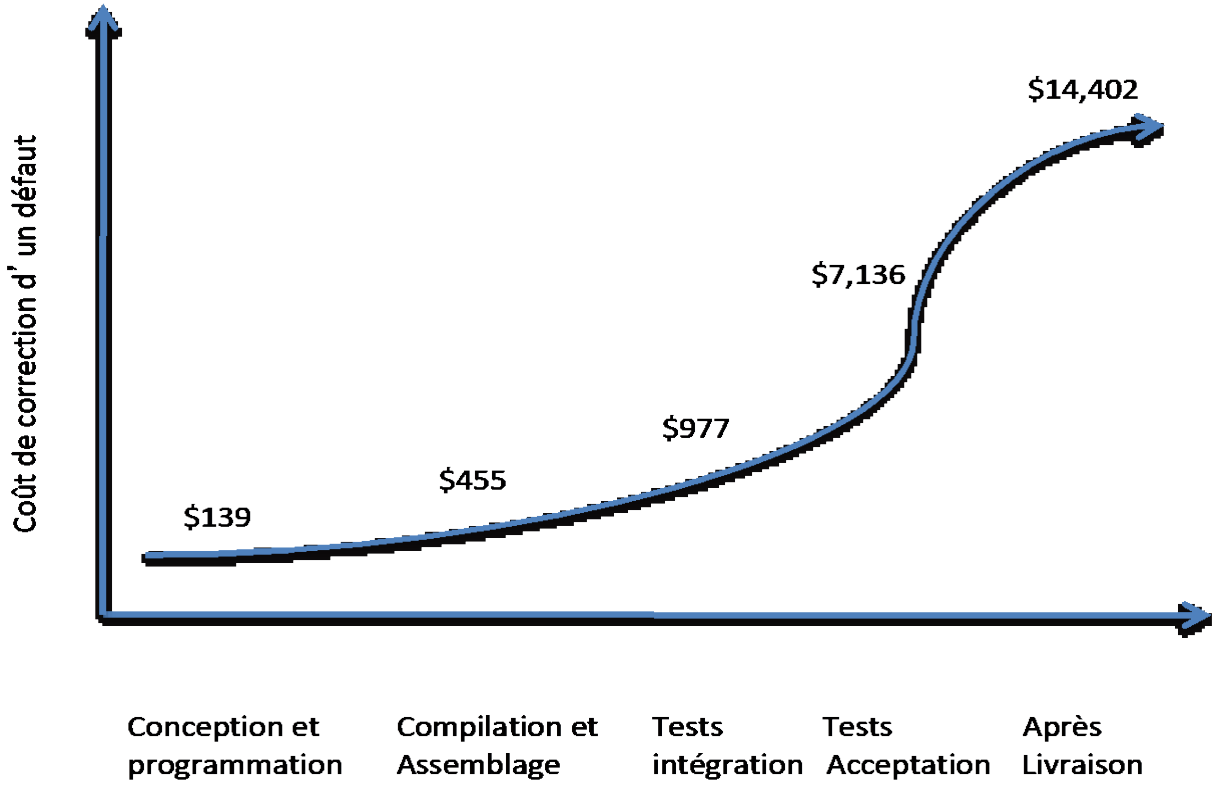
© Alain April, 2020 - Tous droits réservés

Coût de la qualité – les compromis



© Alain April, 2020 - Tous droits réservés

Coût de la propagation des erreurs



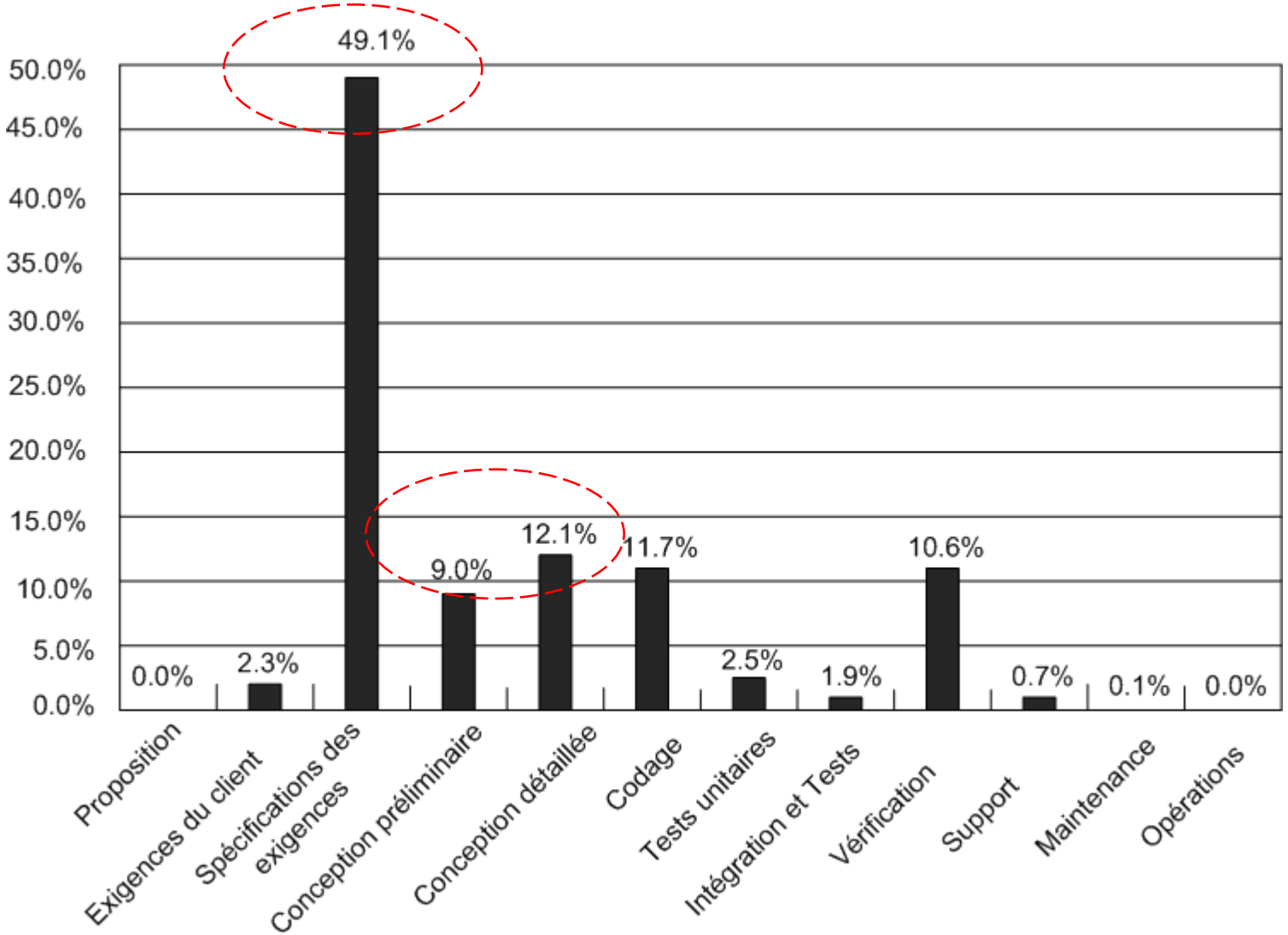
(Boehm 2001)

© Alain April, 2020 - Tous droits réservés

Injection des défauts pendant le développement

Dans quelle phase sont injectées la grande majorité des erreurs logicielles?

Injection des défauts pendant le développement



(Selby, 2007)

© Alain April, 2020 - Tous droits réservés

Constat

- 70% des erreurs sont faites avant la phase de codage !
- Est-Est-ce que les coûts de corrections de celles-celles-ci sont trop élevés pour ne pas faire les activités d'assurance qualité, telles que V&V, inspection, etc.?

Couts de correction

Couts de correction

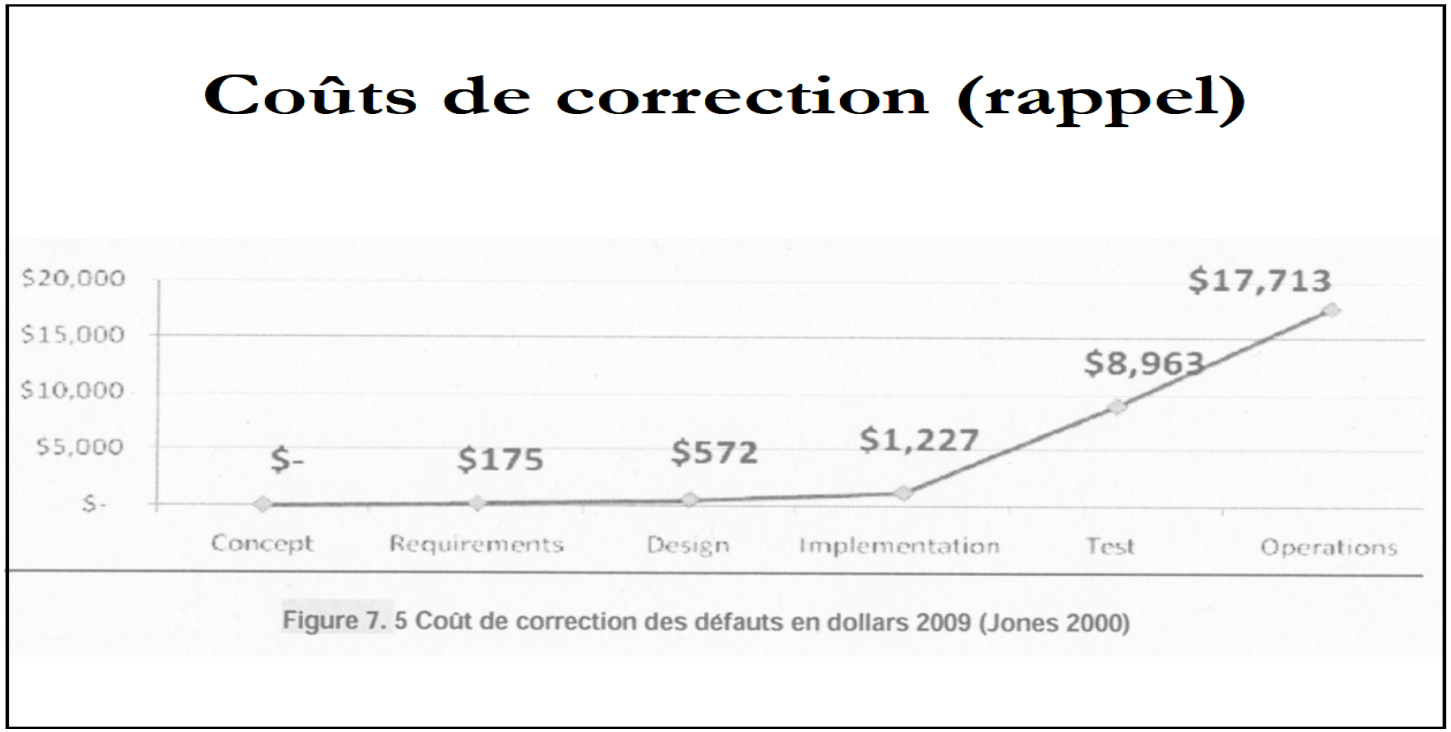


Figure 7. 5 Coût de correction des défauts en dollars 2009 (Jones 2000)

Qualité du logiciel

Définition

- the degree to which a set of inherent characteristics fulfills requirements (ISO 9001)
- Pour qu'un logiciel soit de qualité, il doit:
- Satisfaire les exigences spécifiées
- Satisfaire les besoins du client ou des utilisateurs
- Respecter le budget et le délai de livraison

Qu’est ce que la qualité

La qualité couvre tous les aspects du logiciel

- Processus de développement et de maintenance
- Le code source
- La documentation

Contrôle de qualité

- Un ensemble d’activités conçu pour évaluer la qualité d’un produit.

  L’objectif est de ne pas livrer des produits qui n’atteignent pas le niveau souhaité de qualité.

- Le contrôle de qualité se fait normalement lorsque le produit est terminé.

Assurance vs contrôle de qualité

Assurance vs contrôle de qualité?

Contrôle de qualité

- Produit fini, contrôlé avant sa livraison

Assurance qualité

- Activités faites tout au long du processus de développement

Ces activités :

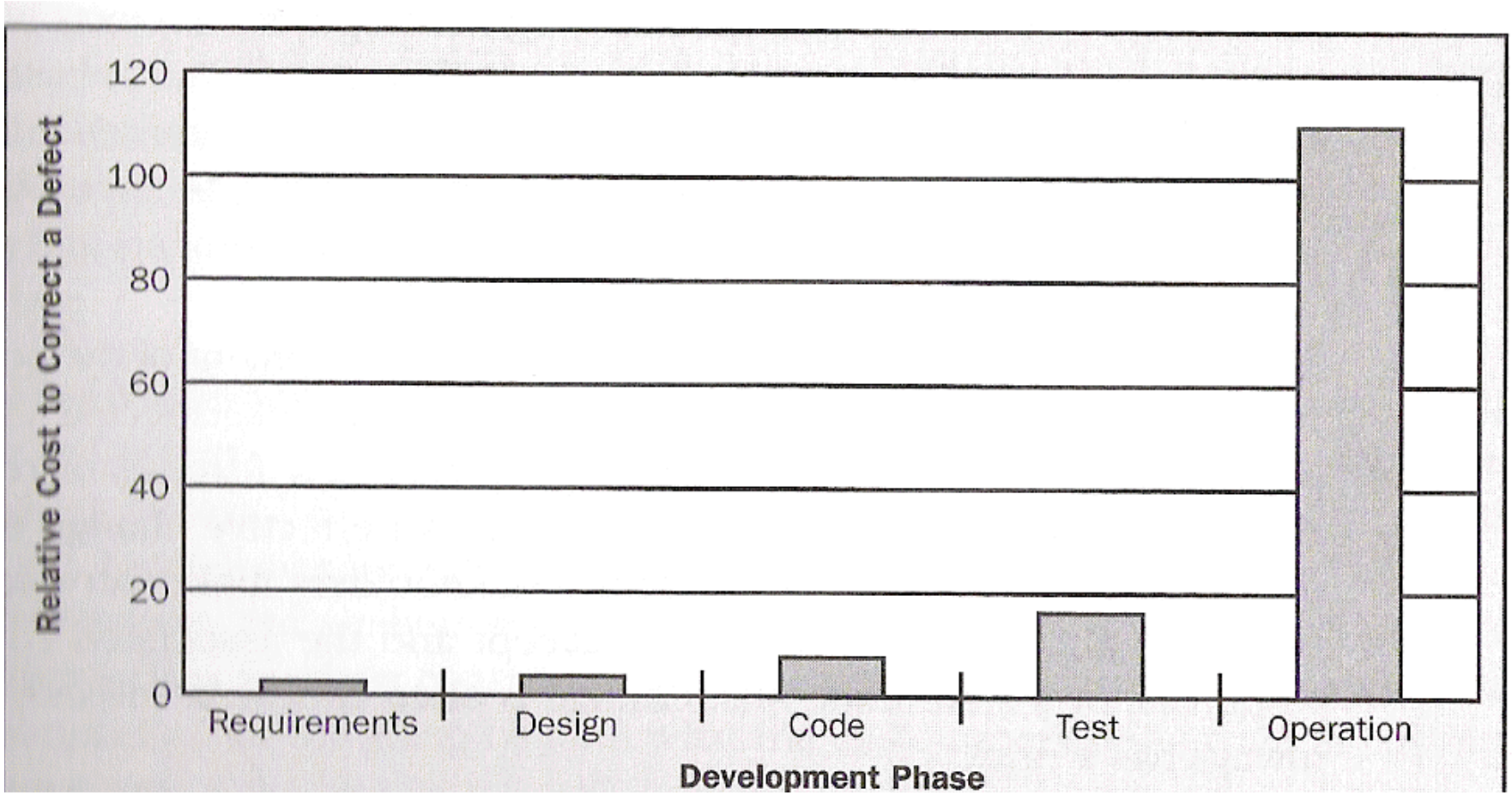
- Préviennent les erreurs
- Les détectent et les corrigent

NB: Le contrôle de qualité est une activité de l'assurance qualité

Activités d'AQ

- Validation et vérification
- Revue des exigences
- Inspection technique
- Tester les exigences
- Valider les exigences avec critères d'acceptation
- Revue de code

Coût des erreurs?



Coût de l'erreur

- Si une erreur au niveau des exigences coute 1 $ à corriger au moment des exigences, elle coutera:
  - 3 à 4 $, si découverte au moment de la conception
  - 100 $ à corriger, après déploiement [Boehm 1981, Grady 1999, Haskins 2004]

V&V

- Le but de la vérification et de la validation est d'aider l'entreprise de développement à bâtir la qualité dans le logiciel tout au long de son cycle de vie (IEEE-(IEEE-1012)

Objectifs de la validation

S'assurer que:

- Les besoins représentent **fidèlement** les fonctionnalités et attributs de qualité attendus des parties prenantes
- Les besoins du système découlent *logiquement* des exigences d'affaires, exigences système, règles d'affaires, etc.
- Les besoins soient ***complètes***, ***faisables***, et ***vérifiables***
- Toutes les besoins soient ***nécessaires***, et l'ensemble est ***suffisant*** pour rencontrer les objectifs d'affaires
- Les différentes représentations des besoins soient ***cohérentes***
- Les besoins constituent une bonne base pour commencer la conception et le codage

Vérification vs *validation*

- Vérification: Les besoins (ou document) ont été construits correctement
  - Building the system ***right***
- Validation: les Besoins (ou document) correspondent aux attentes des parties prenantes
  - Builiding the ***right*** system
- Les deux sont reliées

V&V

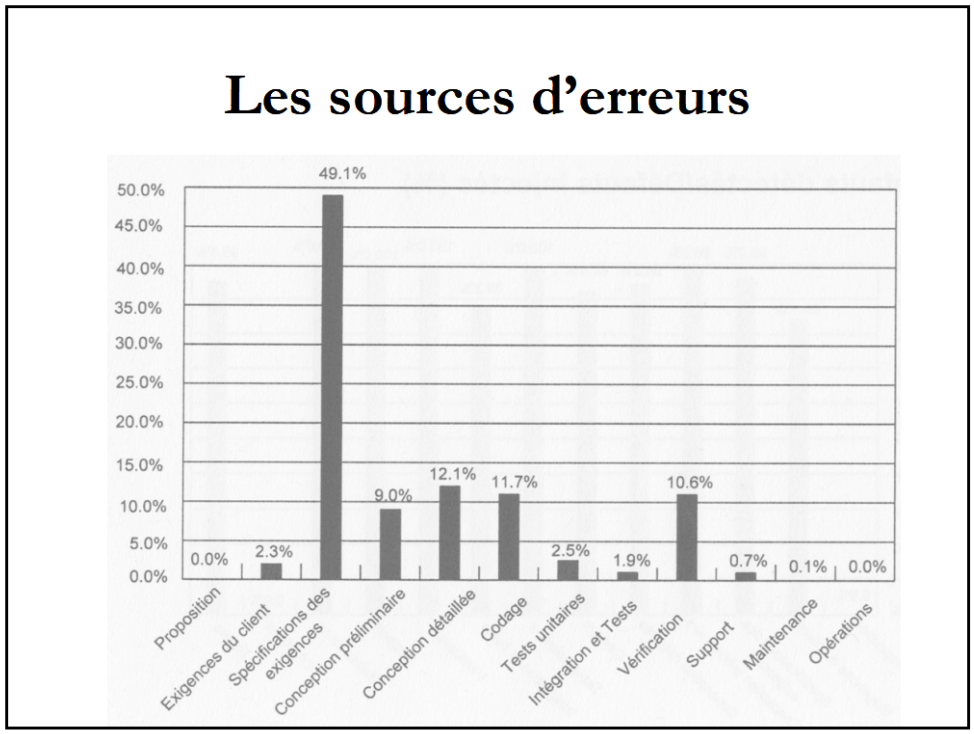
- **Techniques qui permettent de:**
  - Localiser et corriger les erreurs;
  - Vérifier et valider les produits intermédiaires;
  - Déterminer si le logiciel satisfait aux exigences et aux besoins du client

V&V

Quand faire la V&V et pourquoi?

V&V

- #!''


Les sources d'erreurs
50.0%
45.0%
40.0%
35.0%
30.0%
25.0%
20.0%
15.0%
10.0%
5.0%
0.0%
0.0%
2.3%
49.1%
9.0%
12.1%
11.7%
2.5%
1.9%
10.6%
0.7%
0.1%
0.0%
Proposition
Exigences du client
Spécifications des exigences
Conception préliminaire
Conception détaillée
Codage
Tests unitaires
Intégration et Tests
Vérification
Support
Maintenance
Opérations

Constat

- 70% des erreurs sont faites avant la phase de codage !
- Est-Est-ce que les coûts de corrections de celles-celles-ci sont trop élevés pour ne pas faire les activités de V&V?

Coûts de correction

Coûts de correction

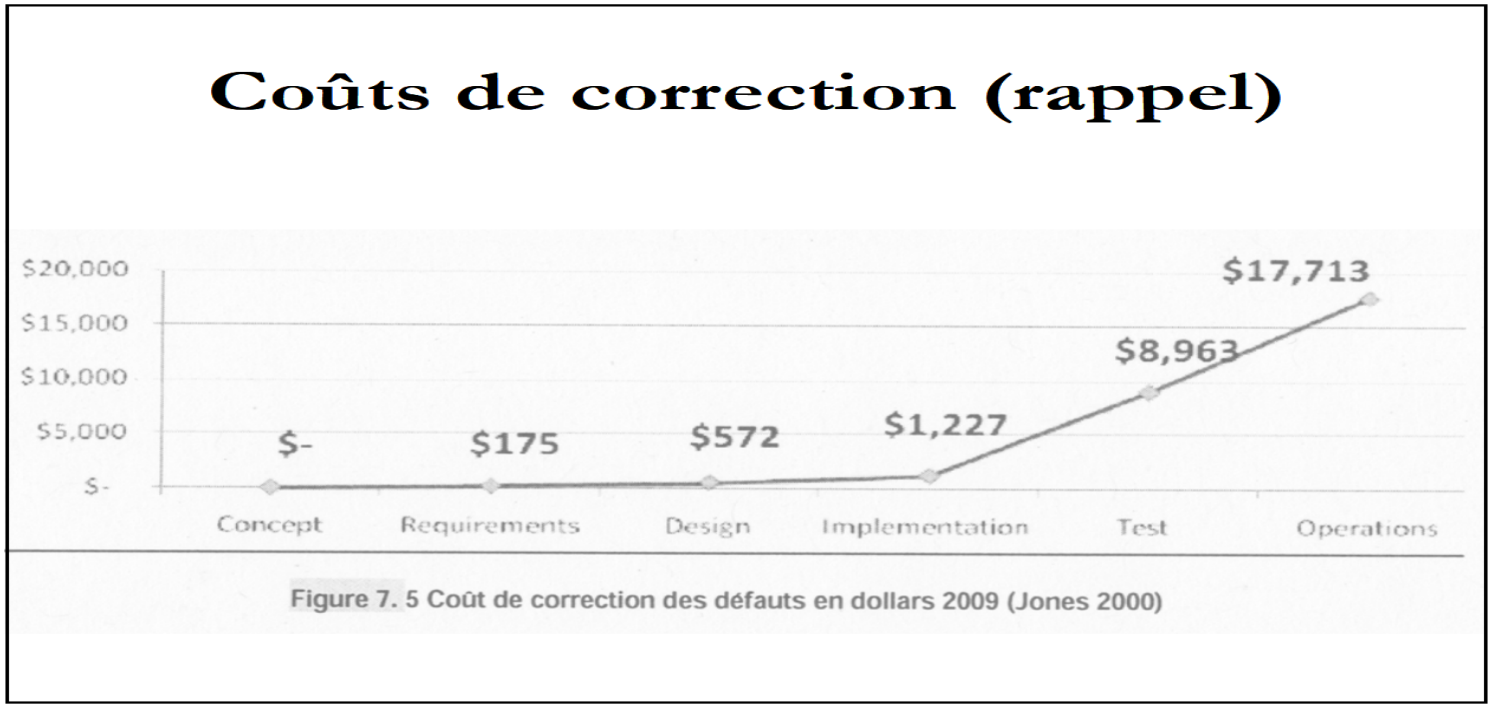


Figure 7. 5 Coût de correction des défauts en dollars 2009 (Jones 2000)

Étude des cas

Dans le cadre de votre projet qui se réalise en 3 sprint, pour lequel on fait recours pour chaque sprint à toutes les activités du cycle de développement de logiciels. Que doit-on faire pour réduire au max le nombre de défauts logiciels dès la phase de spécification?

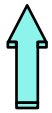
Étude des cas (suite)

Vous êtes en intervention dans un organisme de développement de logiciels. Vous faites votre enquête sur les processus de développement et vous constatez qu'il n'y aucun cout associé aux activités d'assurance qualité. Lors d'une rencontre de suivi avec votre PO, vous lui soulignez votre constatation. A votre grande surprise, il vous répond que ces activités ne servent à rien et qu'elles ajoutent des coûts inutiles aux projets. De plus, il souligne que les projets réalisés sont trop petits pour faire ce type d'activités. Ces dernières (activités) sont réservées plutôt à la **Boeing 737 Max** qu'à son entreprise. Les tests sont donc suffisants, selon lui. Comment allez-vous convaincre votre PO que les activités d'assurance qualité peuvent être très avantageuses pour son entreprise et lui faire renverser sa croyance à ce sujet ?

La culture de l'ingénierie logicielle

- Selon Wiegers une culture saine est composée des éléments suivants :
  - L'engagement **personnel** de chaque chef de projet à créer des projets de qualité en appliquant systématiquement les pratiques efficaces de génie logiciel;
  - l'engagement de **l'organisme** par les gestionnaires de tous les niveaux à fournir un environnement dans lequel la qualité logicielle est un facteur fondamental de succès et qui permet à chaque développeur de réaliser ce but;
  - l'engagement de tous les **membres** de l'équipe à améliorer continuellement les processus qu'ils emploient et à améliorer continuellement les produits qu'ils créent.

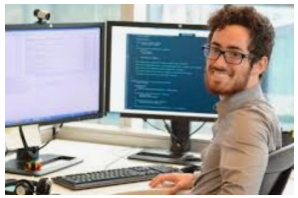




Engagement

Implication



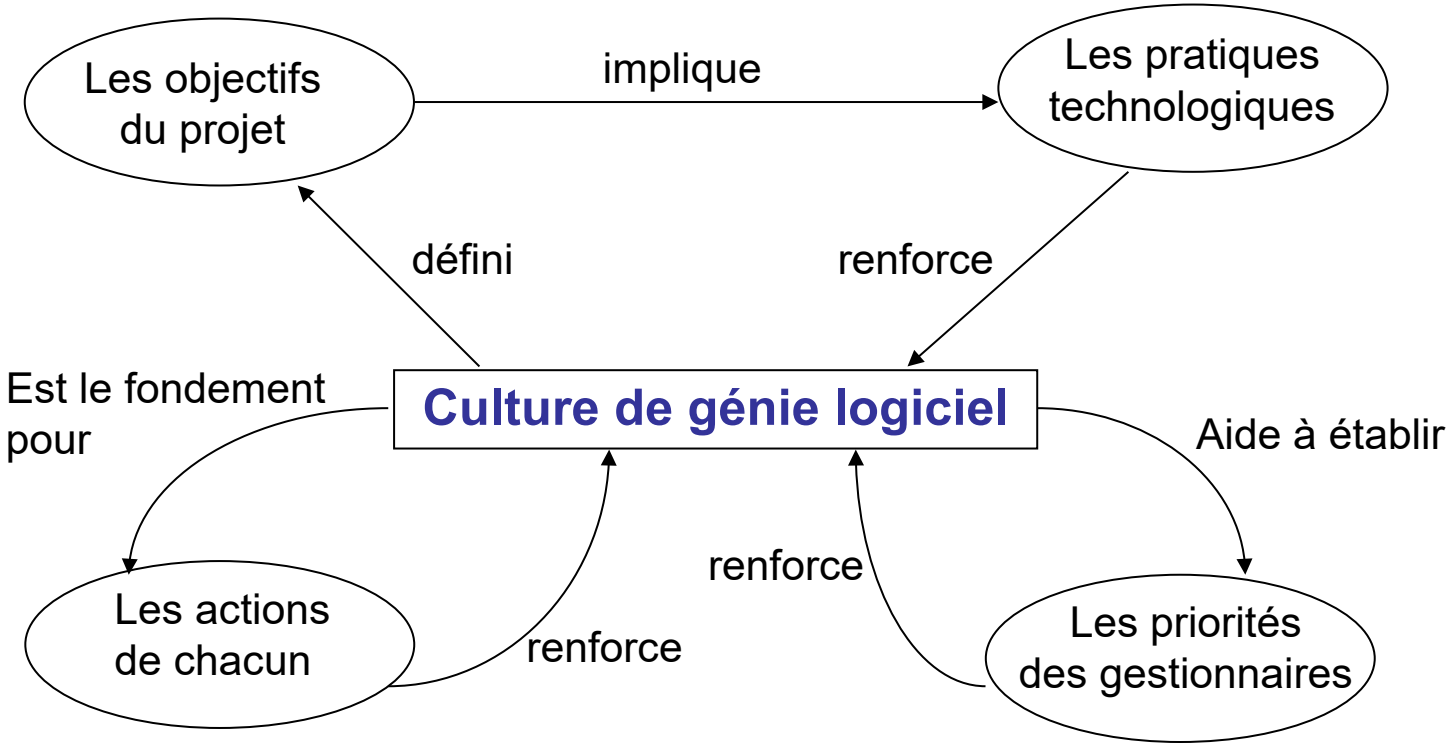


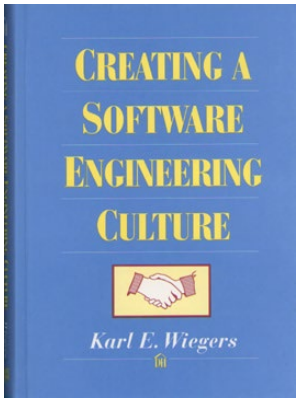
(Wiegers 1996)

© Alain April, 2020 - Tous droits réservés

La culture de l'ingénierie logicielle

- Les interactions entre les comportements, les activités, les priorités et les décisions d'un individu et d'un organisme.





(Wiegers 1996)

© Alain April, 2020 - Tous droits réservés

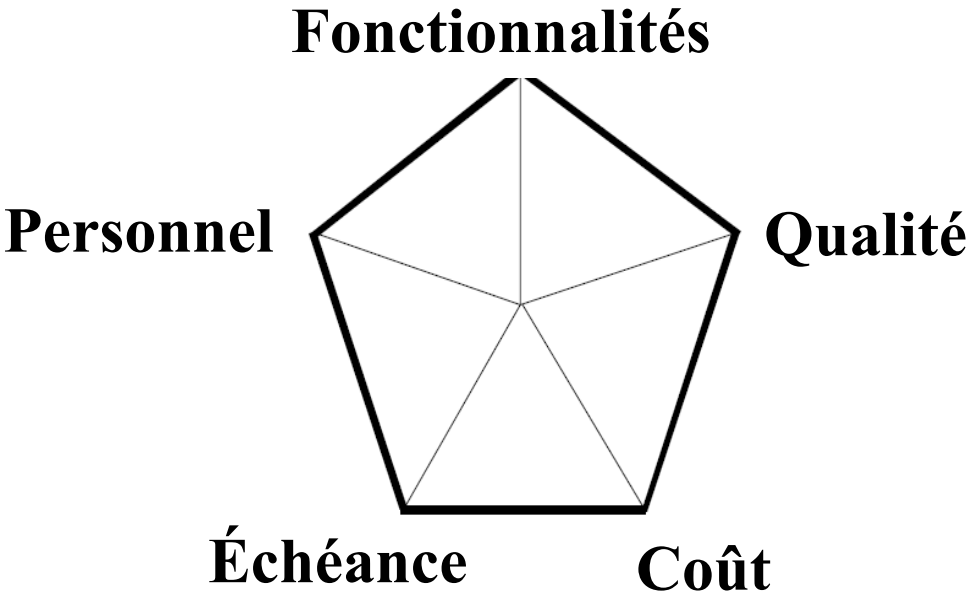
Les principes culturels en génie logiciel

1. Ne jamais laisser votre patron ou votre client vous inciter à faire un mauvais travail;
2. Les personnes doivent sentir que le travail qu'ils effectuent est apprécié;
3. L'éducation continue est la responsabilité de chaque membre d'équipe;
4. La participation du client est le facteur le plus critique de la qualité logicielle;
5. Votre plus grand défi est de partager la vision du produit final avec le client;
6. L'amélioration continuelle de votre processus de développement logiciel est possible et essentielle;
7. Les procédures de développement logiciel peuvent aider à établir une culture commune des bonnes pratiques;
8. La qualité est la première priorité; la productivité à long terme est une conséquence normale de qualité;
9. Tâcher que ce soit un pair, plutôt qu'un client, qui trouve un défaut;
10. Une clé de la qualité logicielle est de réitérer beaucoup de fois sur toutes les étapes de développement excepté le codage: on ne code qu'une fois;
11. Contrôler les rapports d'erreurs et les demandes de changement est essentiel à la qualité et la maintenance;
12. Si vous mesurez ce que vous faites, vous pouvez apprendre à le faire mieux;
13. Faites ce qui semble raisonnable; ne recourez pas aux dogmes;
14. Vous ne pouvez pas changer tout en même temps. Identifiez les changements qui rapporteront les plus grands avantages, et commencez à les mettre en application dès lundi prochain.

© Alain April, 2020 - Tous droits réservés

(Wiegers 1996)

4.4 - Les cinq dimensions d'un projet logiciel et les compromis



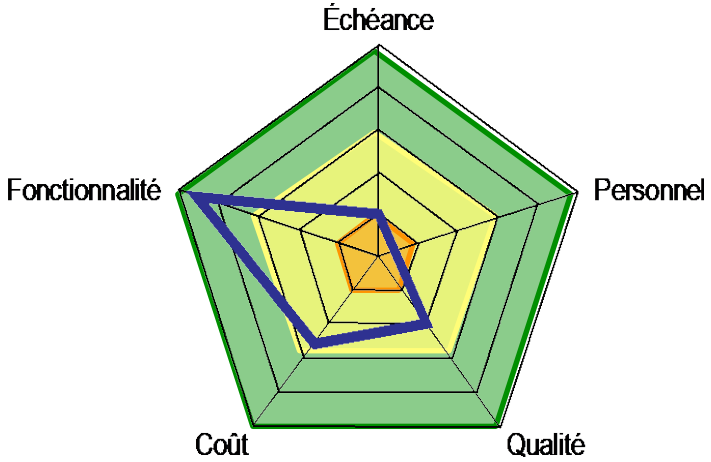
- Trois états (rôles) pour chaque dimension:
  - un objectif (*driver*) est une raison spécifique et importante pour laquelle le projet doit être réalisé. Il n'y a pas de flexibilité;
  - une contrainte est un facteur externe qui n'est pas sous le contrôle du chef de projet. Il y a très peu de flexibilité;
  - un degré de liberté (ce qui n'est pas objectif ou contrainte). Grande flexibilité.

© Alain April, 2020 - Tous droits réservés
(Wiegers 1996)

Les cinq dimensions d'un projet logiciel et les compromis

- Il faut comprendre et possiblement négocier les dimensions avec le client
- Graphe avec une échelle graduée de 10 valeurs où 0 est une contrainte et 10 est la liberté totale.

  - La taille de l'équipe (personnel) est fixe: 1
  - L'échéance ne doit pas être dépassée par beaucoup: 2
  - L'équipe a la liberté concernant les fonctionnalités à implanter : 9
  - Le niveau de qualité de la première version est flexible: 4
  - Le coût étant principalement associé aux dépenses en personnel : 5



© Alain April, 2020 - Tous droits réservés

La négociation d'un projet logiciel

Dimension	Objectif	Contrainte	Degré de liberté
Couts			20 % d'écart acceptable
Fonctionnalités			60 % à 90 % des fonctionnalités de priorité 1 doivent être livrées dans la version 1.0
Qualité			La version 1.0 peut contenir jusqu'à 5 défauts majeurs connus
Échéance	Version 1.0 livrée dans 4 mois		
Personnel		4 personnes	

© Alain April, 2020 - Tous droits réservés

Conclusion

- La qualité est une priorité en génie logiciel
- Non négociable en agile
- Adoption des bonnes pratiques pour bâtir la qualité
- Activités d'AQ sont effectuées tout au long du processus
- Contrôle de qualité se fait à la fin de chaque itération